METODOLOGÍA DE SISTEMAS II

Trabajo Práctico: Entrega Continua y Resiliencia del Software

Parte 1: Integración Continua (CI)

1. Filosofía de Trabajo: Explique la importancia de la filosofía **"Fail-Fast"** dentro de un Pipeline de CI. ¿Cómo beneficia esta práctica a la agilidad del equipo y al ahorro de recursos técnicos?

2. Anatomía del Pipeline (Stages): Para un proyecto profesional, describa qué validaciones se realizan en las siguientes etapas y qué tipo de riesgo mitiga cada una:

- **Lint & Static Analysis:**
- **Unit Tests:**
- **Security Scan:**

3. Pipeline as Code: ¿Qué significa que el archivo de configuración (YAML) sea el "cerebro" del pipeline y por qué es una buena práctica que resida dentro del mismo repositorio de código?

Parte 2: Versionado Semántico (SemVer)

4. El Lenguaje del Cambio: Usted debe asignar un nuevo número de versión a un sistema que actualmente está en la **v2.4.1**. Indique a qué versión pasaría en los siguientes tres escenarios independientes:

- **Escenario A:** Se corrige un error en el cálculo de impuestos que no cambia la forma en que los usuarios usan el sistema.
- **Escenario B:** Se añade una nueva funcionalidad de "Exportar a PDF" que es totalmente compatible con las funciones actuales.
- **Escenario C:** Se elimina una función antigua y se cambia la forma de autenticarse, lo que requiere que otros sistemas que dependen del suyo realicen cambios para no romperse.

5. Tags y Disparadores: Explique técnicamente cómo se vincula el uso de **Tags** en Git con la automatización del despliegue a producción.



Parte 3: Entrega y Despliegue Continuo (CD)

6. Entrega (Delivery) vs. Despliegue (Deployment): ¿Cuál es la diferencia crítica entre ambos conceptos en cuanto a la **intervención humana**? Mencione un caso de negocio donde sea preferible la "Entrega Continua" sobre el "Despliegue Continuo".

7. La Gestión de Entornos: Explique el flujo que sigue el código a través de los entornos de **Dev**, **Staging** y **Prod**. ¿Qué función específica cumple el entorno de *Staging* antes del paso final?

8. El Principio de Inmutabilidad: Defina qué es un **Artefacto Inmutable** y por qué es vital para la seguridad del sistema que no se vuelva a compilar el código cuando se mueve entre distintos entornos.

Parte 4: Resiliencia y Mitigación de Riesgos

9. El Mecanismo de Rollback: Ante una falla crítica en el entorno real, explique:

- ¿En qué consiste un **Rollback** a nivel de infraestructura?
- ¿Cómo ayuda el haber trabajado con **Tags de SemVer** para que este proceso sea casi instantáneo?

10. Estrategias Progresivas: Compare las estrategias **Blue-Green Deployment** y **Canary Deployment**.

- ¿En qué se diferencian respecto al manejo del tráfico de usuarios?
- ¿Cuál de las dos permite probar el código con un grupo reducido de usuarios reales antes del lanzamiento total?